```
# Install torch (only for Colab)
!pip install torch

import torch
import torch.nn as nn
import torch.optim as optim
import random

# ----------------------------
# DATA
# ----------------------------
text = "hello help hello hero hello hill hello world hello nlp"

chars = list(set(text))
char2idx = {ch:i for i,ch in enumerate(chars)}
idx2char = {i:ch for ch,i in char2idx.items()}

data = [char2idx[c] for c in text]

# ----------------------------
# HYPERPARAMETERS
# ----------------------------
vocab_size = len(chars)
embed_size = 64
hidden_size = 128
seq_length = 20
lr = 0.003
epochs = 10

# ----------------------------
# MODEL
# ----------------------------
class CharRNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_size)
        self.rnn = nn.LSTM(embed_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, x, h=None):
        x = self.embed(x)
        out, h = self.rnn(x, h)
        out = self.fc(out)
        return out, h

model = CharRNN()

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=lr)

# ----------------------------
# TRAINING
# ----------------------------
print("Training started...\n")

for epoch in range(epochs):
    total_loss = 0

    for i in range(0, len(data)-seq_length):
        x = torch.tensor(data[i:i+seq_length]).unsqueeze(0)
        y = torch.tensor(data[i+1:i+seq_length+1]).unsqueeze(0)

        optimizer.zero_grad()

        # IMPORTANT FIX: no hidden reuse
        out, _ = model(x, None)

        loss = criterion(out.view(-1, vocab_size), y.view(-1))
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    print(f"Epoch {epoch+1}, Loss: {total_loss:.4f}")
```

```
# ----------------------------
# TEXT GENERATION
# ----------------------------
def generate(model, start="h", length=200, temp=1.0):
    model.eval()

    input = torch.tensor([[char2idx[start]]])
    result = start

    h = None

    for _ in range(length):
        out, h = model(input, h)

        probs = torch.softmax(out[0,-1]/temp, dim=0).detach()
        idx = torch.multinomial(probs, 1).item()

        result += idx2char[idx]
        input = torch.tensor([[idx]])

    return result

print("\n\nGenerated Text (Temp = 0.7):")
print(generate(model, temp=0.7))

print("\nGenerated Text (Temp = 1.0):")
print(generate(model, temp=1.0))

print("\nGenerated Text (Temp = 1.2):")
print(generate(model, temp=1.2))
```

```
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cpu)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.25.2)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
Training started...

Epoch 1, Loss: 37.2977
Epoch 2, Loss: 16.7562
Epoch 3, Loss: 13.0320
Epoch 4, Loss: 10.8220
Epoch 5, Loss: 9.8847
Epoch 6, Loss: 9.3287
Epoch 7, Loss: 9.1026
Epoch 8, Loss: 8.1547
Epoch 9, Loss: 7.9730
Epoch 10, Loss: 7.9385


Generated Text (Temp = 0.7):
hero hello hero hello hill hello hill hello world hello help hello hill hello hill hello world hello hill hello world hello hill

Generated Text (Temp = 1.0):
hill hello world hello hill hello hill hello world hello world hello world hello world hello hero hello world hello nlp hello hi

Generated Text (Temp = 1.2):
held world hello hill hello world hello world help hello world hello world hello hero hero hello hill hello hill hello hill hell
```

```
import torch
import torch.nn as nn
import math

# ----------------------------
# SAMPLE DATA
# ----------------------------
sentences = [
    "i love nlp",
    "transformers are powerful",
    "nlp is interesting",
    "deep learning is fun"
]

# Build vocabulary
```

```
words = list(set(" ".join(sentences).split()))
word2idx = {w:i for i,w in enumerate(words)}
idx2word = {i:w for w,i in word2idx.items()}

# Convert sentences to tensor (pad to same length)
max_len = max(len(s.split()) for s in sentences)
inputs = []

for s in sentences:
    tokens = [word2idx[w] for w in s.split()]
    while len(tokens) < max_len:
        tokens.append(0)  # padding
    inputs.append(tokens)

inputs = torch.tensor(inputs)

# ----------------------------
# POSITIONAL ENCODING
# ----------------------------
def positional_encoding(seq_len, d_model):
    pe = torch.zeros(seq_len, d_model)
    for pos in range(seq_len):
        for i in range(0, d_model, 2):
            pe[pos, i] = math.sin(pos / (10000 ** (i / d_model)))
            if i+1 < d_model:
                pe[pos, i+1] = math.cos(pos / (10000 ** (i / d_model)))
    return pe

# ----------------------------
# MODEL
# ----------------------------
embed_dim = 32
num_heads = 2

embedding = nn.Embedding(len(words), embed_dim)
x = embedding(inputs)

# Add positional encoding
pe = positional_encoding(x.shape[1], embed_dim)
x = x + pe

# Transformer Encoder
encoder_layer = nn.TransformerEncoderLayer(
    d_model=embed_dim,
    nhead=num_heads,
    dim_feedforward=64,
    batch_first=True
)

encoder = nn.TransformerEncoder(encoder_layer, num_layers=1)

# Forward pass
output = encoder(x)

# ----------------------------
# PRINT RESULTS
# ----------------------------
print("Input Tokens:\n", inputs)
print("\nContextual Embeddings Shape:", output.shape)
print("\nSample Embedding:\n", output[0])
```

```
Input Tokens:
 tensor([[ 1,  9,  2,  0],
        [ 3,  8,  7,  0],
        [ 2,  4, 10,  0],
        [ 5,  6,  4,  0]])

Contextual Embeddings Shape: torch.Size([4, 4, 32])

Sample Embedding:
 tensor([[-0.2157, -1.6328,  0.3013,  0.1071,  0.3983,  0.3881, -0.6011,  0.6077,
         -0.2186,  0.5771, -0.1484,  1.1921, -0.5299,  1.6842,  0.7641,  0.1769,
         -0.7662, -0.6267,  0.8875,  0.4278,  0.8245,  2.5860, -0.2805,  0.4313,
         -1.3464, -1.2000, -0.3463,  0.3765, -0.1341,  0.5515, -1.8007, -2.4346],
        [-0.5502,  0.0665, -0.9250,  0.0892,  0.0877, -0.0869,  0.5609,  2.5097,
          0.7700,  1.4856, -0.3980,  2.0561, -0.5155, -0.4750, -2.0124, -0.0329,
         -0.3172,  0.6875, -0.7937, -0.5263, -0.3853,  0.5147,  0.3986,  1.2023,
         -2.1944, -0.5108,  0.3888,  1.2982, -1.0706, -0.5382,  0.0074, -0.7909],
        [ 1.3737, -0.8186, -0.8407, -0.6300,  0.7652, -0.7534,  0.1177,  0.3397,
```

```
          -1.0594,  0.1837, -2.3890,  1.6886, -1.8435,  0.0731,  0.8383, -0.9677,
           0.8876,  0.9562, -0.6565, -0.7397, -1.5363,  0.3657,  0.1893,  1.7259,
          -0.9438,  1.2523,  0.4058,  0.2768,  0.6458,  0.9439, -0.1387,  0.2880],
         [ 0.2348, -0.8660, -1.3053,  0.3823, -0.3499,  0.1128,  1.1035, -0.6453,
          -0.7680, -0.5424, -1.3629,  1.2228, -0.1732,  0.0896,  0.5042,  1.1810,
          -1.9405, -1.0933,  0.4195,  0.8971, -0.4209,  0.5962, -0.5291,  1.5919,
          -1.4934,  2.1511,  0.7304, -0.0942, -1.4710,  1.0883, -0.4417,  1.1916]],
        grad_fn=<SelectBackward0>)
```

```
import torch
import math

def scaled_dot_product_attention(Q, K, V):
    d_k = Q.size(-1)

    # Step 1: Raw scores
    scores = torch.matmul(Q, K.transpose(-2, -1))

    # Step 2: Scale
    scaled_scores = scores / math.sqrt(d_k)

    # Step 3: Softmax
    weights = torch.softmax(scaled_scores, dim=-1)

    # Step 4: Output
    output = torch.matmul(weights, V)

    return scores, scaled_scores, weights, output


# ----------------------------
# TEST WITH RANDOM INPUTS
# ----------------------------
torch.manual_seed(0)

Q = torch.rand(2, 4)
K = torch.rand(2, 4)
V = torch.rand(2, 4)

scores, scaled_scores, weights, output = scaled_dot_product_attention(Q, K, V)

# ----------------------------
# PRINT RESULTS
# ----------------------------
print("Raw Scores:\n", scores)
print("\nScaled Scores:\n", scaled_scores)
print("\nAttention Weights:\n", weights)
print("\nOutput:\n", output)
```

```
Raw Scores:
 tensor([[0.7958, 0.2353],
        [1.0721, 0.7228]])

Scaled Scores:
 tensor([[0.3979, 0.1176],
        [0.5361, 0.3614]])

Attention Weights:
 tensor([[0.5696, 0.4304],
        [0.5436, 0.4564]])

Output:
 tensor([[0.6908, 0.8496, 0.2626, 0.5370],
        [0.6903, 0.8526, 0.2688, 0.5524]])
```